

ESCAPE

Jeu d'aventure sous forme de texte

1. Introduction

Le projet du module de *programmation impérative 1 (PAM2)* consiste à rédiger *un jeu en langage C* jouable dans le terminal. Le programme *escape.c* est un jeu d'aventure sous forme de texte. Dans ce jeu, un étudiant (Guy) diplômé s'est endormi dans l'université lors de la soirée de sa remise des diplômes, et se retrouve enfermé lorsqu'il se réveille après plusieurs heures de sieste. Le but du jeu est de le faire sortir de l'université. J'ai choisi de créer ce jeu, car plusieurs exercices des TP (*GameShell du TP1, jeu_nul.c du TP2*) m'ont beaucoup plu : concevoir un code plus complexe, basé sur ces jeux simples, m'a attiré dès le début.

2. Installation et exécution du programme

Le programme fonctionne sur toutes les machines Linux. Il devrait aussi être fonctionnel dans toutes machines comprenant un compilateur C. L'installation/compilation se fait de la manière suivante : `gcc escape.c`. Pour exécuter le jeu, faire : `./a.out`.

3. Présentation du jeu

Le but du jeu est de pouvoir sortir de l'université. L'utilisateur peut commencer à agir après l'introduction du jeu. Il dispose de **5 lieux** pour organiser ses recherches : d'autres salles de l'université ; le hall ; le service administratif ; l'amphithéâtre Euler ; la sortie (poste de contrôle). Des connaissances en mathématiques seront requises. Vous vous laisserez guider par l'obtention des indices et des codes afin d'obtenir les éléments nécessaires pour avancer dans votre quête. Il est possible de revenir plusieurs fois sur un lieu, bien qu'il est vivement recommandé de les explorer **dans l'ordre**.

4. Organisation du code

Aucune variable globale n'a été utilisée dans ce code. Le programme est notamment découpé en trois sous-parties de fonctions, pour une meilleure compréhension de leurs utilités. Elles sont aussi mises en commentaires dans le code.

Menus et introduction

- void menu/menu_g210 : ces fonctions affichent les menus d'affichage du jeu.

- void intro et int deroule : la première est une longue introduction du contexte du jeu, la deuxième contient le déroulé du jeu.

Lieux

- int autres_salles/hall/administratif/euler/sortie et intro_sortie : les cinq lieux pour réaliser les recherches (*intro_sortie* a été créée pour rendre la fonction *sortie* moins longue en terme de lignes).

Fonctions « techniques »

- void feuille_gauche/feuille_droite/triangle_gauche/triangle_droite : affichent les bouts de feuille et leurs triangles respectifs.

- int saisir_tableaux/comparer_tab_admin/comparer_tab_sortie : permettent de saisir des données sur un tableau et de les comparer avec les vraies valeurs.

5. Bugs et TODO

Le jeu dans sa version actuelle est compilable et ne semble pas présenter de bug. Or, faute de connaissances et de temps, je n'ai pas pu réaliser ou j'ai abandonné les idées suivantes :

- Faire en sorte que, dans sortie, le poste soit verrouillé par la clé obtenue dans le service administratif. Il est vrai qu'elle n'a pas d'utilité dans ce programme.

- L'uniformisation des fonctions de comparaison : en essayant cela, les mots de passe de n'importe quelles valeurs pouvaient passer. J'ai dû garder les fonctions de comparaisons séparées.

6. Remerciements

J'ai réalisé ce projet par moi-même, mais je compte remercier mon professeur de TP (*M. Rousselin*) pour m'avoir aiguillé dans quelques défauts du code. C'est aussi, en grande partie, grâce aux exercices des TP (GameShell, jeu_nul/le donjon, menu de l'exo 4 du TP7, triangle d'étoiles en valeurs, comparaisons/saisies de tableau) que j'ai pu mener à bien mon jeu.

7. Conclusion

Au final, ce jeu s'est avéré plus difficile à programmer. Pourtant je me suis beaucoup amusé à le créer, et surtout, uniquement par le langage C (honnêtement, je croyais qu'il était impossible de faire un jeu là-dessus !).

C'est une excellente initiative pour mobiliser nos connaissances.

Ce programme mobilise la plupart des notions étudiées (les boucles (for/while/do), les booléens, les tableaux, les chaînes de caractères, les fonctions, les structures).